Блок №1

4 Задачи

Храбров Артём Алексеевич

Задача 1

Использую скользящее окно длины 3 для того чтобы идти по массиву. Изначально длина максимального участка 0. А начальный индекс текущего участка также равен 0. Проходим окном по массиву до тех пор, пока все элементы окна не будут равны. Как только это произошло до вычисляем текущий индекс конца интервала. Используя индекс начала и конца сравниваем длину максимального и текущего участка и выбираем наибольший из них. Делаем так до конца массива и выводим индексы наибольшего участка. Сложность решения получилась O(n)

```
#include <iostream>
using namespace std;

void push(int array[], int element) {
  array[0] = array[1];
  array[1] = array[2];
  array[2] = element;
}

bool checkEqual(int a[]) { return (a[0] == a[1]) && (a[1] == a[2]); }

void compareToMaxRangeAndSwap(int maxRange[], int start, int end) {
  if ((maxRange[1] - maxRange[0]) < (end - start)) {
    maxRange[0] = start;
    maxRange[1] = end;
  }
}

int main() {
  int n;
  int a[3];
  int tmp;

  int startIdx = 0;
  int endIdx = 0;
  int maxRange[2] = {0, 0};
  cin >> n;

  for (int i = 0; i < n; i++) {
    cin >> tmp;
    push(a, tmp);
    if (checkEqual(a)) {
      compareToMaxRangeAndSwap(maxRange, startIdx, endIdx);
      startIdx = i - 1;
      endIdx = i;
    } else {
      endIdx = i;
    }
  }
  compareToMaxRangeAndSwap(maxRange, startIdx, endIdx);
  cout << maxRange[0] + 1 << " " << maxRange[1] + 1 << endl;
```

```
  return 0;
}
```

Задача 2

Задча похожа на задачу скобочной последовательности. Будем считать парой символы одной буквы разного регистра. С помощью стека будем записывать все символы которые встретились. Также проверяем является ли втреченный символ парой с вершиной стека. Если да, то удаляем эту пару со стека и вычисляем индекс животного и индекс ловушки, записываем их в ответ. В итоге проверяем пуст ли стек, если да то значит все пары нашлись. Сложность моего решения O(n) т.к. достаточно одного прохода по строке для результата.

```
#include <iostream>
#include <map>
#include <stack>
#include <vector>
using namespace std;

int abs(int n) {
  if (n < 0)
    return -n;
  else
    return n;
}

bool isPair(char a, char b) {
  if (abs(a - b) == 32)
    return true;
  else
    return false;
}

bool isAnimal(char a) {
  return ('a' <= a) && ('z' >= a);
}

void printAns(bool ans, map<int, int> ansMap) {
  if (ans) {
    cout << "Possible\n";
    for (const auto& [key, animal] : ansMap) {
      cout << animal << " ";
    }
  } else {
    cout << "Impossible\n";
  }
}

int main() {
  char c;
  stack<char> stk;
  stack<int> indexes;
  int catcherCnt = 0;
  int animalCnt = 0;
  map<int, int> ans;

  while (cin >> c) {
    int idx;
    if (isAnimal(c)) {
      idx = ++animalCnt;
```

```
    } else {
      idx = ++catcherCnt;
    }

    if (!stk.empty() && isPair(c, stk.top())) {
      if (isAnimal(c)) {
        ans[indexes.top()] = idx;
      } else {
        ans[idx] = indexes.top();
      }
      stk.pop();
      indexes.pop();
    } else {
      stk.push(c);
      indexes.push(idx);
    }
  }
  printAns(stk.empty(), ans);
  return 0;
}
```

Задача 3

Чтобы хранить состояние переменных при перемещении между блоками удобно использовать map где ключ это переменная а значение это стек значений переменной. При изменении переменной в блоке кладем в стек новое значение, а при выходе удаляем с верхушки. Для отслеживания какие переменные менялись в каком блоке используем стек из векторов. В вектор будем записывать измененные в блоке переменные и при выходе из блока по ним проходить, чтобы восстановить им значение. Для каждого блока будет свой вектор. Сложность решения $O(N^2)$. Если внутри блока изменить все введенные переменные, то после выхода из блока нужно будет пройти по ним всем чтоб восстановить значение. Количество скобок в большом объеме ничтожно мало по сравнению с основными строками, поэтому их не считаем.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <stack>
#include <vector>
using namespace std;

bool isNumber(string s) {
  for (size_t i = 0; i < s.length(); i++) {
    if (!((s[i] >= '0' && s[i] <= '9') || s[i] == '-')) {
      return false;
    }
  }
  return true;
}

vector<string> parseData(string s) {
  vector<string> data = {"", ""};
  for (size_t i = 0; i < s.length(); i++) {
    if (s[i] == '=') {
      data[0] = s.substr(0, i);
      data[1] = s.substr(i + 1, s.length());
      break;
    }
  }
  return data;
}

int main() {
```

```
  map<string, stack<int>> variables;
  stack<vector<string>> changes;
  string line;

  vector<string> newVector;
  changes.push(newVector);

  while (getline(cin, line)) {
    if (line == "{") {
      vector<string> newVector;
      changes.push(newVector);
    } else if (line == "}") {
      vector<string> ch = changes.top();
      for (size_t i = 0; i < ch.size(); i++) {
        variables[ch[i]].pop();
      }
      changes.pop();
    } else {
      int value;
      vector<string> data = parseData(line);

      if (isNumber(data[1])) {
        value = stoi(data[1]);
      } else {
        if (variables.find(data[1]) == variables.end() ||
variables[data[1]].empty()) {
          value = 0;
        } else {
          value = variables[data[1]].top();
        }
        cout << value << endl;
      }

      variables[data[0]].push(value);
      changes.top().push_back(data[0]);
    }
  }
  return 0;
}
```

Задача 4

Так как значение k может быть огромное, то задача не может быть решена обычным циклом симуляции дней. Если изучить условие то видно, как после какждого дня число бактерий либо будет бесконечно расти, что приведет к ограничению их числа из-за контейнера и d бактерий после каждого дня, либо число бактерий будет уменьшаться и станет равно 0. Если значение обратится в 0 то это не займет много времени, поэтому можно использовать цикл. А если будет бесконечно расти, то при совпадении числа бактерий 2 дня подряд их число останется постоянным и ответ будет равняться этому же числу. Так как число бактерий очень быстро устремляется либо в 0 либо к числу d, то при соблюдении условий задачи алгоритм никогда не отработает полностью при больших k. С учетом скорости компилируемых языков сложность алгоритма можно считать O(1).

```
#include <iostream>
using namespace std;

int main() {
  long long a, b, c, d, k;
  cin >> a;
  cin >> b;
```

```
  cin >> c;
  cin >> d;
  cin >> k;

  long long f = a;
  long long previous = 0;

  for (long long i = 1; i < k + 1; i++) {
    f = f * b;
    f = f - c;
    if (f < 0) {
      f = 0;
      break;
    }

    if (f > d) {
      f = d;
      break;
    }

    if (f == previous) {
      break;
    }
    previous = f;
  }
  cout << f;

  return 0;
}
```